Institut Universitaire de la Côte – 3IL / 3IAC
Cycle Ingénieur informatique – 3IL_4 : EADL / JA
Normale Semestre I (2026-2027)

Machine Learning et Modélisation des Données

Rapport – Industrialisation MLOps d'un modèle de prédiction des impayés (default_risk)

Enseignant : Dr KUATE Alain François
Étudiante : Djamila Diallo Beyas
Dépôt Git : https://github.com/djamilabeyas/mlops-impayes
Octobre 2026

Partie I – Connaissance du cours

Questions de cours

R1. Définir le MLOps et présenter deux de ses objectifs.

Le MLOps (Machine Learning Operations) est l'ensemble des pratiques, outils et processus qui automatisent et fiabilisent le cycle de vie complet d'un modèle d'apprentissage automatique : préparation des données, entraînement, validation, déploiement, surveillance et réentraînement. Il applique au Machine Learning les principes du DevOps et de l'ingénierie des données.

- **Industrialiser la mise en production :** passer d'un modèle expérimental à un service fiable, déployé de façon automatisée et répétable.
- **Garantir la reproductibilité et la traçabilité :** pouvoir reconstruire exactement un modèle (données, code, paramètres) et savoir quelle version a produit quelle décision.

R2. Expliquer la différence entre Machine Learning et MLOps.

Le Machine Learning est une discipline scientifique : il consiste à concevoir et entraîner un modèle qui apprend à partir des données et à mesurer sa performance (choix d'algorithme, entraînement, évaluation). Le MLOps est une discipline d'ingénierie : il consiste à exploiter ce modèle en production dans la durée (versioning, automatisation, déploiement, surveillance, réentraînement). Le ML produit le modèle ; le MLOps le met en production et le maintient performant.

R3. Définir CI, CD et CT.

- **CI – Intégration continue :** à chaque modification poussée sur le dépôt, le code est automatiquement vérifié (analyse statique, tests unitaires, validation des données) afin de détecter les erreurs au plus tôt.
- **CD – Livraison / déploiement continu :** une fois les tests passés, le modèle et son API sont automatiquement empaquetés (image Docker) et déployés vers un environnement de test puis de production.
- **CT – Entraînement continu :** le modèle est réentraîné automatiquement sur des données récentes, de façon planifiée ou lorsqu'une dégradation ou une dérive est détectée, puis validé avant toute promotion.

R4. Citer deux outils MLOps et préciser le rôle de chacun.

- **MLflow :** suivi des expériences (paramètres, métriques, artefacts) et registre de modèles (versions, statut staging / production).
- **DVC :** versioning des jeux de données et des pipelines de données, en lien avec les commits Git.

Consolidation des connaissances

R5. Identifier quatre difficultés auxquelles l'entreprise est confrontée.

1. **Absence de mise en production :** le modèle ne tourne que sur l'ordinateur du développeur ; il est inaccessible aux autres systèmes et dépend d'une seule personne.
2. **Mises à jour manuelles :** lentes, sources d'erreurs et non reproductibles.
3. **Aucune surveillance après déploiement :** une dégradation des performances ou une panne ne peut pas être détectée.
4. **Absence de versioning et de traçabilité :** données et modèles non versionnés, retour arrière et explication des décisions impossibles.

R6. Proposer une chaîne MLOps comportant au moins cinq étapes.

1. **Ingestion et versioning des données :** extraction automatique des factures, retards et ancienneté ; versioning avec DVC.
2. **Préparation et validation :** nettoyage, traitement des valeurs manquantes, feature engineering, contrôles de qualité (schéma, valeurs aberrantes).
3. **Entraînement et suivi des expériences :** entraînement scikit-learn, enregistrement des paramètres et métriques dans MLflow.
4. **Évaluation et validation :** précision, rappel, F1, AUC ; le modèle n'est accepté que s'il dépasse les seuils fixés et fait au moins aussi bien que le modèle en production.
5. **Enregistrement :** inscription dans le registre MLflow avec le statut staging.
6. **Déploiement :** API FastAPI conteneurisée avec Docker, déployée automatiquement par la chaîne CI/CD.
7. **Surveillance :** latence, taux d'erreurs, dérive des données, F1 sur les résultats réels ; alertes et déclenchement du réentraînement.

R7. Choisir trois outils adaptés au projet et justifier leur utilisation.

Outil	Rôle dans le projet	Justification
MLflow	Suivi des expériences et registre de modèles	Intégration native avec scikit-learn ; conserve versions, métriques et artefacts ; permet le retour arrière.
Docker	Empaquetage du modèle et de l'API	Environnement identique du poste de développement au serveur ; déploiement portable et reproductible.
GitHub Actions	Automatisation CI/CD/CT	Intégré au dépôt Git ; enchaîne tests, entraînement, validation, construction et déploiement à chaque modification.

R8. Expliquer comment Docker et une API faciliteraient l'exploitation du modèle.

- **API :** le modèle devient un service accessible par le réseau. Le système de facturation envoie les données d'un client à un point d'entrée (POST /predict) et reçoit le risque d'impayé en temps réel, sans dépendre du poste du développeur.
- **Docker :** l'image contient la version exacte de Python, des bibliothèques et du modèle. Elle s'exécute à l'identique sur tout serveur, se duplique pour absorber la charge et se remplace par la version précédente en cas de problème.

R9. Proposer deux indicateurs de surveillance et une action en cas de dégradation.

- **F1-score sur les résultats réels :** calculé lorsque le paiement effectif du client est connu ; alerte si la valeur passe sous le seuil fixé.
- **Dérive des données (PSI) :** écart entre la distribution actuelle des variables et celle de l'entraînement ; alerte si PSI > 0,2.
- **Action :** déclencher une alerte, analyser la cause, réentraîner le modèle sur des données récentes, le valider puis le déployer progressivement ; en attendant, revenir à la version précédente si nécessaire.

R10.1. Interpréter les trois observations.

- **Latence de 100 ms à 900 ms :** problème technique d'infrastructure (surcharge, ressources insuffisantes, traitement plus lourd) et non de qualité du modèle ; les applications appelantes sont ralenties.

- **Forte évolution de la distribution des montants :** dérive des données (data drift) ; les clients actuels ne ressemblent plus à ceux de l'entraînement (évolution des tarifs, des offres, inflation).
- **F1 de 0,88 à 0,65 :** dégradation réelle des performances, liée à la dérive des données et probablement à une dérive de concept ; le modèle rate des impayés ou multiplie les fausses alertes.

R10.2. Proposer trois mesures correctives ou préventives.

1. **Latence :** augmenter les ressources ou le nombre de conteneurs, optimiser le prétraitement, mettre en place une alerte sur la latence p95.
2. **Performance :** réentraîner le modèle sur des données récentes, le valider puis le déployer progressivement (canari), avec retour arrière possible.
3. **Prévention :** surveillance continue de la dérive et du F1 réel avec seuils d'alerte, et réentraînement planifié.

R10.3. Expliquer pourquoi il ne faut pas remplacer automatiquement le modèle dès qu'une dérive est détectée.

- Une dérive des données ne signifie pas forcément une baisse de performance ; il faut d'abord mesurer son impact réel.
- La dérive peut être temporaire (saisonnalité, promotion) ou provenir d'une erreur de données (bug d'ingestion, changement d'unité) ; réentraîner sur ces données dégraderait le modèle.
- Un nouveau modèle n'est pas nécessairement meilleur : il doit être validé et comparé au modèle en production pour éviter une régression.
- Chaque remplacement doit être tracé et approuvé (décisions de crédit sur des clients) ; des remplacements trop fréquents coûtent en calcul et rendent le système instable.

Partie II – Application : mise en production

QCM

Question	Réponse	Énoncé retenu
Q1	B	Ingestion des données → Entraînement du modèle → Déploiement → Monitoring
Q2	B	Les données d'entraînement et les artefacts de modèle
Q3	C	Fixer les graines aléatoires, la version des données, le commit de code et les dépendances
Q4	B	Assurer l'observabilité technique et comportementale du système en production
Q5	B	Gérer les versions des modèles, leur statut (staging/production) et les métadonnées associées

Q6. Composants du pipeline MLOps et outils recommandés

Composant	Rôle dans le projet	Outil	Justification
Ingestion	Chargement du jeu Telecom Credit Scoring Data (500 000 lignes, 15 colonnes) depuis Hugging Face	Python, datasets, pandas	Chargement direct et reproductible du jeu public
Versioning des données	Figer chaque version des données et des étapes de préparation	DVC	Lie une version de données à un commit Git
Validation des données	Présence de la cible, absence de valeurs nulles, tenure_months ≥ 0	Contrôles Python (extensible Pandera)	Bloque le pipeline sur des données corrompues
Préparation / features	Retrait des identifiants (customer_id, phone_number) et de la date constante ; imputation ; encodage one-hot ; cible high = 1, low/medium = 0	scikit-learn Pipeline	Transformation identique à l'entraînement et en production
Entraînement et suivi	HistGradientBoostingClassifier, classes pondérées, graine fixée	scikit-learn, MLflow Tracking	Comparaison des essais, reproductibilité
Validation (gate)	Précision, rappel, F1, AUC, matrice de confusion ; seuils F1 ≥ 0,55 et AUC ≥ 0,72	MLflow, GitHub Actions	Décision automatique et traçable
Registre de modèles	Versions, alias staging / production, métadonnées	MLflow Model Registry	Promotion et retour arrière maîtrisés
Serving	Points d'entrée /predict, /health, /metrics, /drift/check, documentation /docs	FastAPI	Rapide ; génère la documentation OpenAPI
Conteneurisation	Image de l'API avec le modèle, utilisateur non root, healthcheck	Docker, GHCR	Portable et reproductible
CI/CD/CT	Lint, tests, entraînement, validation, build, scan, déploiement ; réentraînement planifié	GitHub Actions, Prefect	Intégré au dépôt ; orchestration du réentraînement
Monitoring	Latence, erreurs, dérive (PSI), F1 réel ; alertes	Prometheus, Grafana	Couvre l'observabilité technique et comportementale

Q7. Data drift et concept drift

Le modèle apprend une relation entre des variables d'entrée X (ancienneté, dépenses, fréquence de recharge, cohérence des paiements, retards) et la cible y (risque d'impayé).

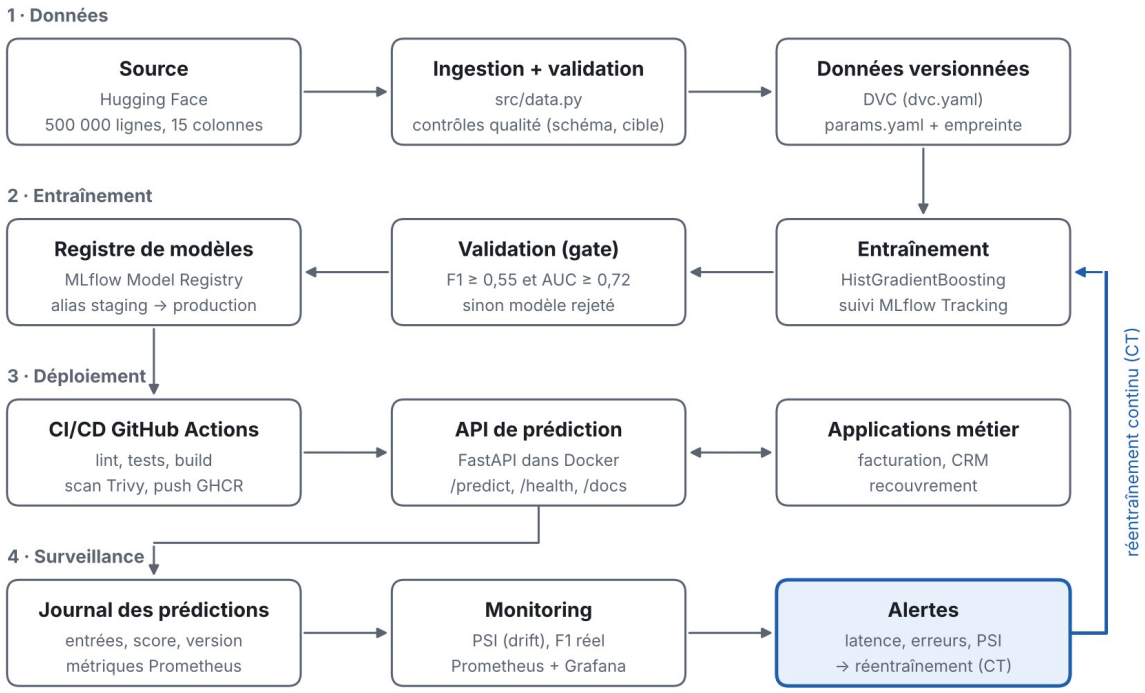
- **Data drift :** la distribution des entrées P(X) change, alors que la relation entre X et y reste la même. Elle se détecte immédiatement en comparant les distributions, sans attendre le résultat réel.
  - Exemple : après le lancement d'une offre data à bas prix, la part des clients récents (tenure_months < 3) passe de 10 % à 40 % et la dépense mensuelle moyenne (avg_monthly_spend_ngn) baisse fortement. Le modèle reçoit des profils qu'il a peu vus à l'entraînement.
- **Concept drift :** la relation entre X et y, P(y | X), change : des profils identiques n'ont plus le même comportement de paiement. Elle ne se détecte qu'avec les résultats réels, par une baisse du F1 ou du rappel.
  - Exemple : une crise économique ou une hausse du coût de la vie entraîne des défauts de paiement chez des clients anciens qui rechargeaient régulièrement ; leurs variables n'ont pas changé mais leur risque réel a augmenté.

Q8. Stratégie de redéploiement sûre

1. **Validation hors ligne :** la nouvelle version (v2) doit franchir la gate (F1 ≥ 0,55, AUC ≥ 0,72) et égaler au moins la version en production (v1) sur un jeu de test récent ; l'API et l'image Docker passent les tests.
2. **Déploiement en staging :** l'image est déployée dans un environnement de pré-production et vérifiée par des tests de fumée (/health, /predict, /docs, /metrics).
3. **Déploiement canari :** v2 reçoit d'abord 5 % du trafic, puis 25 %, 50 % et 100 %, avec observation à chaque palier de la latence, du taux d'erreurs et de la proportion de clients prédits à risque.
4. **Test A/B :** deux groupes de clients tirés au hasard sont servis par v1 et v2 ; on compare sur une durée suffisante le F1 réel, les impayés évités et les faux positifs ; v2 n'est retenue que si l'écart est statistiquement significatif.
5. **Critères de rollback :** retour immédiat à v1, conservée dans le registre, si la latence p95 dépasse 300 ms, si le taux d'erreurs dépasse 1 %, si le F1 réel de v2 est inférieur de plus de 0,05 à celui de v1, ou si la proportion de prédictions « impayé » devient anormale.
6. **Promotion :** après validation à 100 % du trafic, v2 reçoit l'alias production dans le registre MLflow ; v1 est archivée et reste disponible.

Exercice pratique de conception

i. Schéma d'architecture



- **Données :** la source Hugging Face est ingérée et validée (src/data.py), puis versionnée avec DVC et params.yaml.
- **Entraînement :** le modèle est entraîné et suivi dans MLflow, validé par la gate, puis enregistré dans le registre avec l'alias staging.
- **Déploiement :** GitHub Actions teste, construit et scanne l'image Docker, la publie sur GHCR et la déploie ; les applications métier appellent l'API FastAPI.
- **Surveillance :** chaque prédiction est journalisée ; Prometheus collecte les métriques, Grafana les affiche, les alertes déclenchent le réentraînement continu.

ii. Pipeline CI/CD/CT

CI – .github/workflows/ci-cd.yml (à chaque push et pull request)

1. **lint :** analyse statique du code avec ruff.
2. **test :** tests unitaires pytest (entraînement, API, encodage de la cible, calcul du PSI, règle de réentraînement).

CT – entraînement et validation

1. **train :** récupération des données (DVC, sinon Hugging Face), entraînement, enregistrement MLflow ; le modèle et ses métriques sont publiés comme artefact.
2. **validate :** gate automatique F1 ≥ 0,55 et AUC ≥ 0,72 ; en cas d'échec, la chaîne s'arrête et la production n'est pas modifiée.
3. **Réentraînement continu (.github/workflows/ct.yml) :** planifié chaque lundi à 3 h, lancé manuellement ou déclenché par une alerte de dérive (événement drift-alert) ; le

flow Prefect flows/retrain_flow.py contrôle quotidiennement la dérive et lance un entraînement candidat.

CD – construction et déploiement

1. **build :** construction de l'image Docker, scan de vulnérabilités Trivy (échec si faille critique), publication sur GHCR étiquetée par le commit.
2. **deploy-staging :** déploiement de l'image publiée et tests de fumée (/health, /predict, /openapi.json, /docs, /metrics).
3. **deploy-production :** déploiement sur le serveur par SSH, soumis à l'approbation de l'environnement production.

iii. Plan de monitoring

Catégorie	Métrique suivie	Seuil d'alerte	Action
Technique	Latence p95 des requêtes	> 0,3 s pendant 5 min	Ajout de ressources ou de conteneurs, analyse
Technique	Taux d'erreurs HTTP 5xx	> 1 % pendant 5 min	Retour à la version précédente
Données	Dérive par variable (PSI max)	> 0,2 pendant 10 min	Alerte et analyse ; entraînement candidat
Données	Qualité des entrées (schéma, valeurs manquantes)	Colonne absente ou cible nulle	Blocage de l'ingestion
Performance	F1 sur les résultats réels	< 0,50	Lancement du réentraînement (CT)
Métier	Proportion de clients prédits à risque	Écart anormal par rapport à la référence	Analyse et revue du seuil de décision

Mise en œuvre : l'API expose ses métriques sur /metrics (Prometheus) ; les règles d'alerte sont définies dans monitoring/alert_rules.yml ; le tableau de bord Grafana affiche la latence p95, le débit, le taux d'erreurs et le PSI maximal. Le point d'entrée /drift/check compare les dernières prédictions aux données de référence.

Déclenchement du réentraînement : une alerte de dérive (PSI > 0,2) ou un F1 réel inférieur à 0,50 déclenche un entraînement candidat. Le nouveau modèle n'est promu qu'après passage de la gate, déploiement en staging et validation humaine ; sinon le modèle de production est conservé.

iv. Gestion du cycle de vie

Politique de versioning

- Code : Git, chaque modification passe par la chaîne CI.
- Données : DVC (dvc.yaml), paramètres dans params.yaml, empreinte des données enregistrée avec chaque modèle.
- Modèles : MLflow Model Registry (default-risk-model), une version par entraînement validé, alias staging puis production, avec commit, empreinte des données, paramètres et métriques.
- Images : chaque image Docker est étiquetée par le commit qui l'a produite.
- Les versions antérieures sont conservées pour permettre le retour arrière.

Gouvernance des données

- Données personnelles : customer_id et phone_number sont exclus de l'entraînement et de l'API.
- Qualité : contrôles automatiques à l'ingestion ; documentation du jeu et des prétraitements dans le dépôt.
- Accès : secrets de déploiement stockés dans GitHub Secrets ; conteneur exécuté sans droits administrateur.
- Équité : contrôle que le modèle ne pénalise pas un groupe (opérateur, type de compte).

Auditabilité des décisions

- Chaque prédiction est journalisée avec l'horodatage, les entrées, le score et la version du modèle.
- Chaque run MLflow conserve le commit Git, l'empreinte des données, les paramètres, les métriques et la matrice de confusion.
- Toute promotion en production est tracée (approbation de l'environnement GitHub, alias MLflow).

Livrables techniques

Livrable	Contenu
1. Dépôt Git structuré	https://github.com/djamilabeyas/mlops-impayes – src/ (data, train, api, monitoring), notebooks/01_exploration_entrainement.ipynb, tests/, flows/, scripts/, monitoring/, registry/, docs/, README.md
2. Données versionnées	dvc.yaml et dvc.lock (empreintes md5 des données, du code et des paramètres), params.yaml ; prétraitements : cible high = 1, low/medium = 0 ; retrait de customer_id, phone_number, assessment_date ; imputation et encodage one-hot
3. Modèle entraîné et évalué	models/metrics.json, models/meta.json, notebook (matrice de confusion, courbe ROC, importance des variables), artefact « model » de chaque run GitHub Actions, runs MLflow (expérience default-risk)
4. API de serving	src/api.py (FastAPI) : POST /predict, GET /health, /metrics, POST /drift/check ; documentation OpenAPI sur /docs
5. Image Docker	Dockerfile, docker-compose.yml (API, MLflow, Prometheus, Grafana) ; image publiée sur ghcr.io/djamilabeyas/mlops-impayes
6. Pipeline CI/CD/CT	.github/workflows/ci-cd.yml (lint, test, train, validate, build, deploy-staging, deploy-production), .github/workflows/ct.yml, flows/retrain_flow.py
7. Registre de modèles	MLflow Model Registry : default-risk-model version 1, alias staging et production ; export registry/model_registry.json ; artefact « mlflow-registry » de chaque run ; captures docs/captures/mlflow_*.png
8. Dashboard de monitoring	Tableau de bord Grafana monitoring/grafana/dashboards/mlops.json, monitoring/prometheus.yml, monitoring/alert_rules.yml ; captures docs/captures/grafana_dashboard.png et prometheus_alertes.png
9. Rapport	docs/Rapport_MLOps_Impayes.pdf et docs/Rapport_MLOps_Impayes.docx

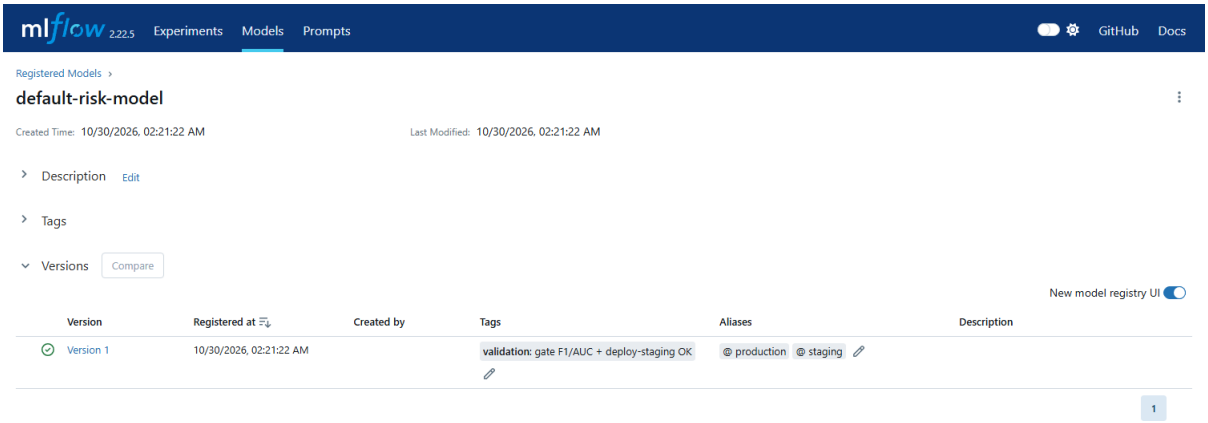
Performances du modèle (jeu de test : 100 000 lignes)

Métrique	Valeur
Précision	0,496
Rappel	0,697
F1-score	0,580
AUC	0,746

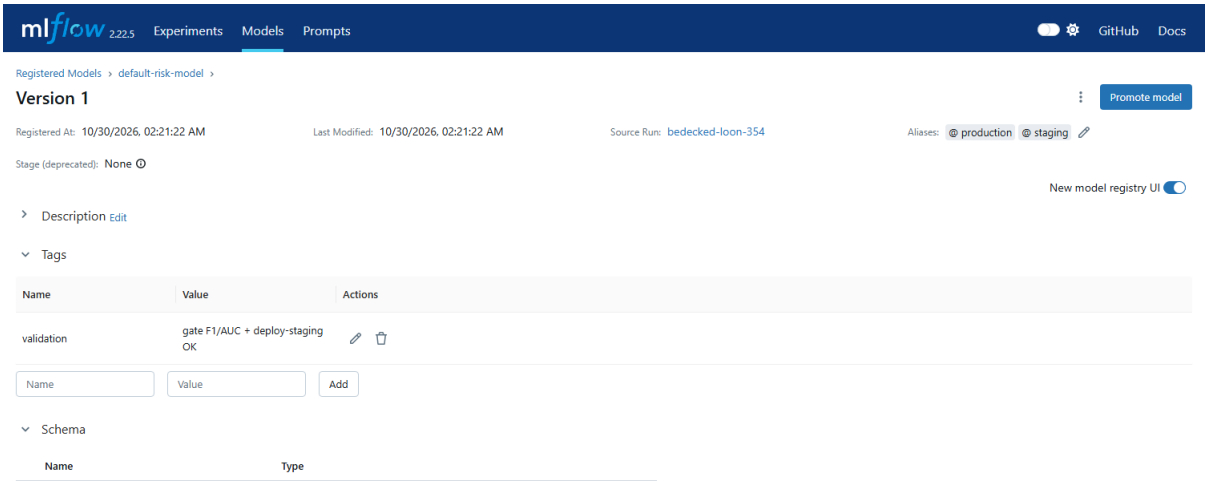
Matrice de confusion

	Prédit : paie (0)	Prédit : impayé (1)
Réel : paie (0)	62 004	15 738
Réel : impayé (1)	6 748	15 510

Registre de modèles MLflow



Registre MLflow : modèle default-risk-model, version 1, alias staging et production

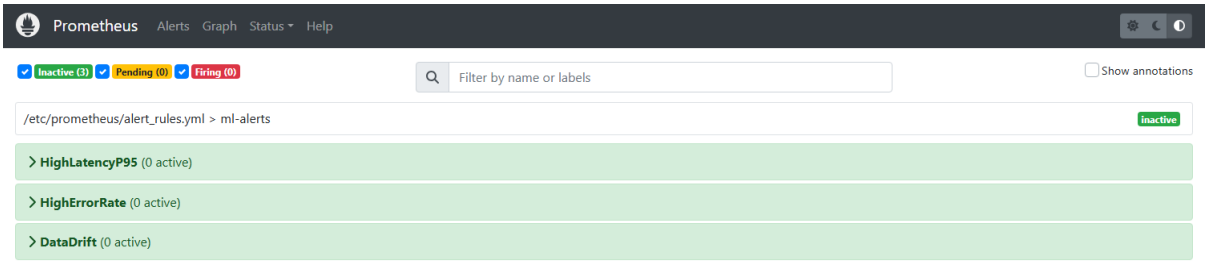


Version 1 : métriques, paramètres et traçabilité (commit, empreinte des données)

Dashboard de monitoring



Tableau de bord Grafana : latence p95, débit, taux d'erreurs, dérive (PSI)



Règles d'alerte Prometheus : latence, erreurs, dérive des données